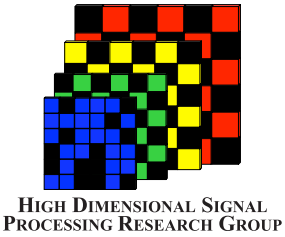


Grupo de Investigación HDSP

Escuela de Ingeniería de Sistemas e Informática, Universidad Industrial de Santander

---

Laboratory Workshop
Deep Learning and Inverse Problems

Workshop overview

Pipeline: images → forward model → training → metrics

Output: Notebook + report with figures, a small metrics table, and discussion.

Learning goals

You will implement small experiments in Python and write answers that explain what you observed (complementing the screenshots).

By the end, you should be able to:

- Describe inverse problems mathematically and recognize common cases,
- Train an MLP for regression and classification and understand its parts,
- Build and train a CNN autoencoder for simple image restoration tasks,
- Reason about performance curves, overfitting, and basic regularization.

Note: Use GPU if available, but every experiment here can run on CPU.

1 From Linear Models to Neural Networks

Deep Learning is a modern way to build *functions from data*. In supervised learning we are given examples $\{(x_i, y_i)\}_{i=1}^{N}$, where x_i is an input and y_i is its desired output. The goal is to construct a model f_θ (with parameters θ) that produces predictions $\hat{y} = f_\theta(x)$ which generalize well to new, unseen inputs. Learning means adjusting the parameters θ so that the model's predictions become progressively closer to the desired outputs across the training examples.

For example, the input x could be a vector of pixel intensities in an image, sensor readings, or features extracted from data; and the output y could represent a quantity we want to predict: a class label (e.g., "cat" or "dog"), a continuous value (such as temperature or depth), or even another image. In all cases, the network learns a mapping from inputs to outputs based on examples. In this first part, we start from the simplest learnable models and gradually add the key ingredient that makes neural networks powerful.

Figure 1: Learning a function from data: A learned image classification model, mapping from an input x to an output y.

A simple baseline: Linear Models and the Perceptron

A linear model predicts $\hat{y} = w^\top x + b$, where w is a weight vector and b is a bias term. Geometrically, this model represents a hyperplane in the input space and can only model linear relationships between inputs and outputs. The basic computation unit of the linear models are called **perceptron**, which computes a linear combination $z = w^\top x + b$ and then applies a non-linear activation function $\phi(\cdot)$, producing $\hat{y} = \phi(z)$.

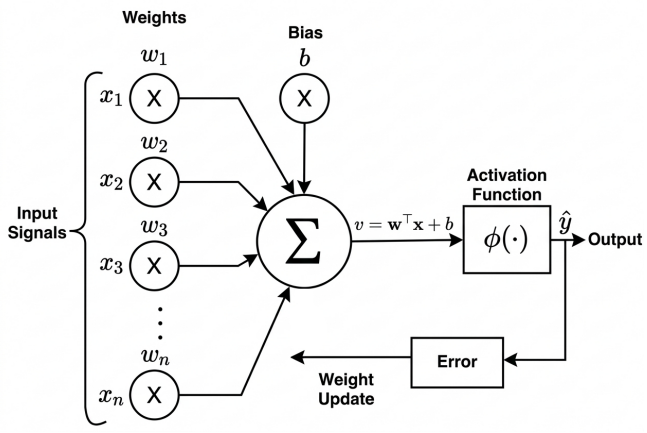


Figure 2: In a linear layer, inputs are weighted, summed with a bias, and passed through an activation function to produce the output. The parameters are updated during training using the error signal.

Without an activation function, stacking several linear layers would still produce a linear mapping, since the composition of linear transformations is itself linear. In other words, this would not increase the expressive power of the model. Activation functions introduce controlled non-linearities, allowing the network to bend, fold, and reshape the input space in ways that a single hyperplane cannot. They determine how strongly neurons respond to different inputs, influence gradient flow during training, and shape the kinds of patterns the network can represent. Choosing an activation function is therefore not merely a technical detail: it directly affects stability, convergence behavior, and the ability of the model to capture complex structures in data, and its selection often depends on the specific task being addressed.

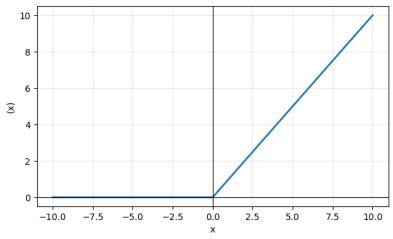
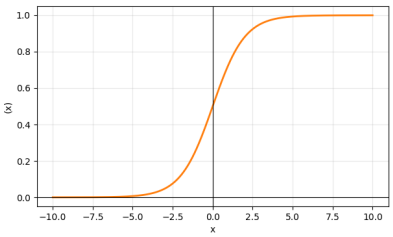
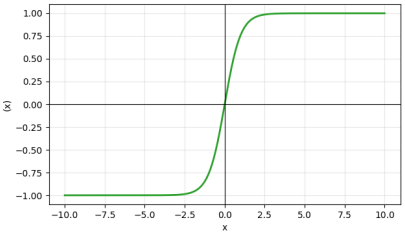
Activation	Formula	Description	Shape
ReLU	$\phi(t) = \max(0, t)$	Simple and widely used; introduces sparsity and avoids saturation for positive values.	
Sigmoid	$\phi(t) = \dfrac{1}{1+e^{-t}}$	Squashes inputs into $(0, 1)$; often used for binary classification outputs.	
Tanh	$\phi(t) = \tanh(t)$	Outputs in $(-1, 1)$; zero-centered but can saturate for large magnitudes.	

Table 1: Common activation functions used in neural networks.

Multilayer perceptron (MLP)

An MLP is a neural network obtained by composing multiple perceptrons. It is organized into an input layer, one or more hidden layers, and an output layer. The input layer receives the data x, the hidden layers apply successive linear transformations followed by non-linear activations, and the output layer produces the final prediction. For example, $h^{(1)} = \phi(W_1 x + b_1) \to h^{(2)} = \phi(W_2 h^{(1)} + b_2) \to \hat{y} = W_3 h^{(2)} + b_3$. By stacking layers in this way, the network increases its capacity, enabling it to model increasingly complex patterns and dependencies present in data.

Although MLPs are commonly used for tabular data, they can also process images by flattening the pixels into a single vector, thereby ignoring their spatial arrangement. However, this is not

the most natural or efficient way to handle image data, since it discards the underlying spatial structure.

Training: Reducing Prediction Error

So far, we have described how a neural network produces a prediction $\hat{y}$ from an input x. The remaining question is how to adjust the parameters so that these predictions become more accurate.

To improve the model, it is necessary to measure how different the prediction $\hat{y}$ is from the desired output y. This discrepancy is quantified by a function, often denoted by $\mathcal{L}$, whose purpose is to measure how well the model is performing. The specific mathematic form of $\mathcal{L}$ depends on the task being solved.

Across a dataset $\{(x_i, y_i)\}_{i=1}^{N}$, we evaluate this discrepancy for all training examples and adjust the parameters θ to make it smaller. In this way, learning can be understood as progressively reducing the overall prediction error on the data. To achieve this reduction in practice, we need a systematic way to modify the parameters in the direction that most effectively decreases $\mathcal{L}$.

Training: Fitting Parameters Using Gradients

Training updates the parameters θ (all weights and biases) to reduce the loss by using the **gradient descent** method:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where η is the learning rate, i.e., a stepsize that controls how large each parameter update is in the direction of the negative gradient. Computing $\nabla_\theta \mathcal{L}$ using the chain rule is what backpropagation provides. In practice, training is organized into **epochs**: each epoch corresponds to one full pass through the training set; training for multiple epochs repeats this process to progressively improve performance.

Question

How is backpropagation used to compute $\nabla_\theta \mathcal{L}$ in an MLP?

During training, we track:

- **Training loss**: Performance on the data used to train the model.
- **Validation loss**: Performance on unseen data used only for evaluation during training. It just serves as guidance.

An important warning sign when comparing these two quantities is *overfitting*: the training loss continues to decrease while the validation loss begins to increase. This indicates that the model is **memorizing** the training data, capturing specific patterns that do not generalize well to new inputs, and therefore cannot be reliably applied to real-world scenarios.

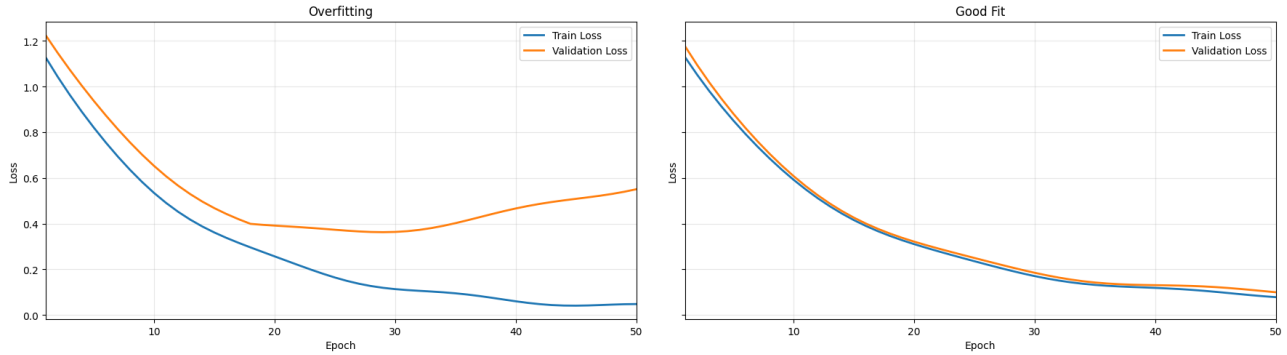


Figure 3: Training vs. validation loss curves showing good fit, and overfitting.

Question

What strategies can be used to prevent overfitting? Briefly explain how they work.

2 Convolutions and CNNs: Deep Learning that respects image structure

Convolutional Neural Networks (CNNs) were developed to address a key limitation of fully connected networks when dealing with images. CNNs are designed to exploit the local structure and geometric regularities present in images, making them more efficient and better suited for visual tasks.

Images as structured data

An image can be represented as a collection of numbers organized on a grid. A grayscale image is a matrix

$$x \in \mathbb{R}^{H \times W},$$

where each entry $x_{i,j}$ corresponds to the intensity at spatial location (i, j). A color image has multiple channels (for example RGB) and can be represented as a tensor

$$x \in \mathbb{R}^{C \times H \times W},$$

where C denotes the number of channels.

Unlike generic vectors, images have an inherent spatial structure: neighboring pixels are related, and their arrangement carries meaningful geometric information. If we simply flatten an image into a long vector and feed it into a fully connected network, this spatial organization is lost. Convolutional Neural Networks (CNNs) are designed to preserve and exploit this spatial structure.

What is a convolution?

A convolution applies a small kernel across the image:

$$(x * k)(i, j) = \sum_{m,n} x(i - m, j - n)\, k(m, n),$$

where x denotes the input image, k is the convolution kernel (or filter), (i, j) indicates the spatial location of the output pixel, and the indices m, n run over the kernel size. Intuitively, the kernel slides over the image and computes local weighted sums. Different kernels detect different patterns (edges, corners, textures).

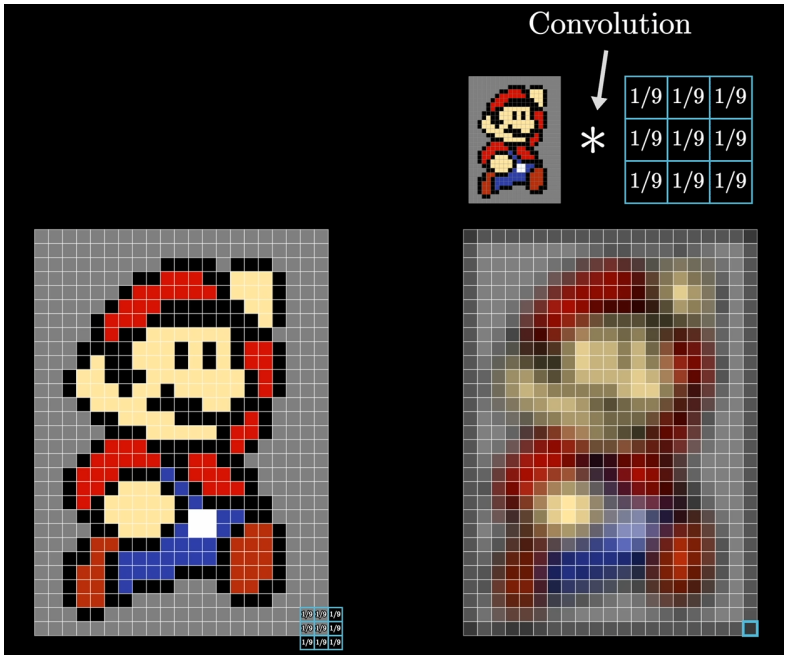


Figure 4: Blur kernel applied to an image.

A typical CNN block consists of: convolution → activation (ReLU) → typically, pooling or normalization. In this pipeline, the convolution kernel weights are the main trainable parameters; normalization layers may also include additional trainable scale and shift parameters, while pooling layers do not introduce new parameters.

Why convolutions are powerful

Convolutions have two major advantages compared to fully connected layers:

- They use **parameter sharing**, meaning that the same kernel (set of weights) is applied across all spatial locations of the image. This drastically reduces the number of parameters and makes the model more efficient and less prone to overfitting.
- They exploit **locality**, so each output value depends only on a small neighborhood of pixels. Early layers therefore learn simple local patterns such as edges or textures, while

deeper layers progressively combine these local features into more complex and larger-scale structures.

Common CNN Architectures

Beyond a single convolutional block, practical CNNs are built by stacking multiple layers in structured ways. Each kernel produces its own output feature map, and stacking these feature maps together forms the output tensor of the layer. The number of kernels (also called the number of output channels) is a design choice of the architecture and determines how many different types of patterns the layer can learn to detect.

As we move deeper into the network, two common operations are used:

- **Increasing the number of channels**: Applying more kernels allows the network to extract a richer variety of features.
- **Downsampling**: Reducing the spatial resolution (for example using pooling or strided convolutions) allows the network to aggregate information over larger regions of the image.

Together, these operations enable the network to build hierarchical representations, moving from simple local patterns to more abstract features. A common architectural pattern in CNNs is the **feature extractor**, where several convolutional layers progressively reduce spatial resolution while increasing the number of channels. This reduction in resolution is often achieved using operations such as strided convolutions or max-pooling. As spatial dimensions shrink and channels increase, the network captures increasingly high-level representations of the input.

One important application of this idea is the **autoencoder** architecture. In this case, an encoder acts as a feature extractor that compresses the input into a high-level representation (also known as a bottleneck), while a decoder restores spatial resolution to produce an output of the same size as the input. The restoration step may involve upsampling operations, such as transposed convolutions or interpolation followed by convolution.

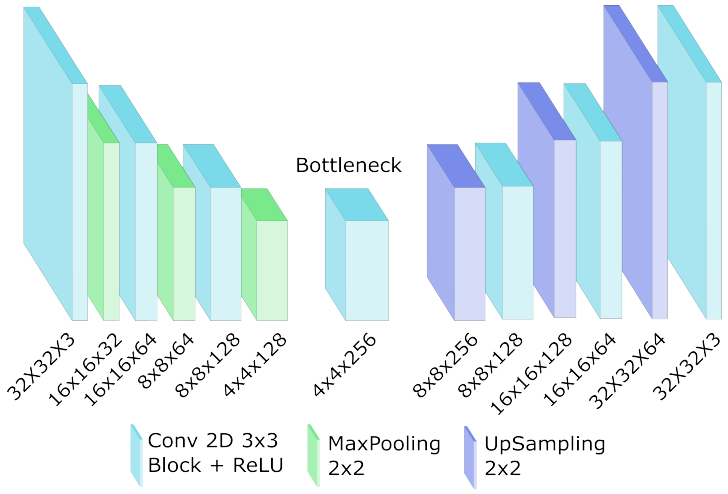


Figure 5: Illustration of a generic encoder–decoder CNN architecture for a 32×32 RGB image.

3 Inverse problems: Why are they challenging?

A large family of tasks in imaging can be written as

$$y = Hx + n,$$

where x is the unknown clean image (usually known as **ground truth**), H is a **forward operator** describing how measurements are formed, n is noise, and y is what we observe (the measurement). The goal is to estimate x from y.

Figure 6: Think of x as the sharp, ideal scene in front of you. When you take a photo while your hand is moving, the camera does not record x directly. Instead, the imaging system (the optics, sensor, and motion) acts as the operator H, producing a blurred image y. Noise n accounts for sensor imperfections and low-light effects. In this view, solving the inverse problem means reconstructing the sharp image you wish you had captured from the blurred measurement your camera actually produced.

Common inverse problems and what H looks like

Problem	Forward Model	Description
Denoising	$y = x + n \quad (H = I)$	The measurement equals the clean image plus noise. The forward operator is the identity.
Deblurring	$y = x * k + n$	The forward operator is a convolution with a blur kernel k. High-frequency details are attenuated, and edges become smoother.
Inpainting	$y = Hx + n$	The forward operator is a masking matrix that removes pixels (sets them to zero or deletes them). The mask is usually known.
Super-resolution	$y = D(Bx) + n$	The operator includes blur B followed by downsampling D. The goal is to recover a high-resolution image from a low-resolution measurement.

Table 2: Common inverse imaging problems and their forward models.

Many inverse problems are **ill-posed**: multiple different x can produce similar y (or exactly the same y). Noise makes it worse, because small perturbations in y can lead to large changes in a naive reconstruction.

Question

Why are many inverse problems *ill-posed*? In particular, why can multiple different x produce the same observation y under the model $y = Hx$?

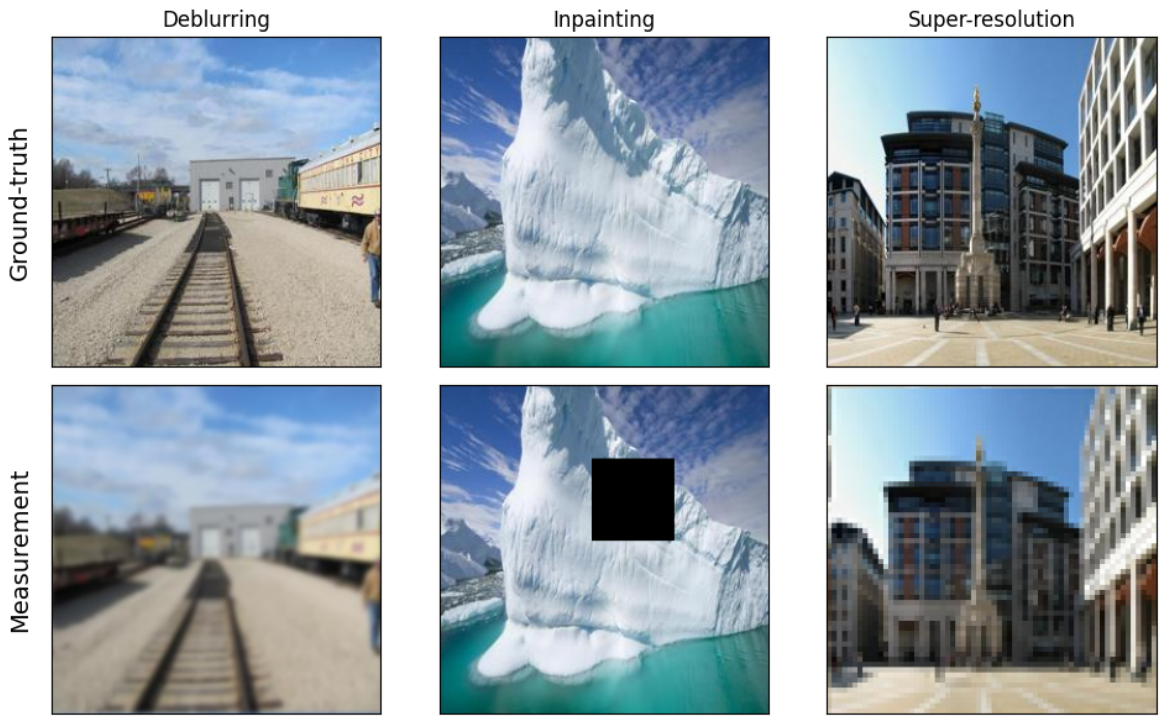


Figure 7: Examples of inverse imaging problems: ground truth (top) and corresponding measurements (bottom) for deblurring, inpainting, and super-resolution.

Learning to solve inverse problems

The inverse model $y = Hx + n$ describes how measurements are produced from an unknown clean image x. In an inverse problem, we observe y and want to recover x. One way to approach this with deep learning is to train a neural network to learn a direct mapping from measurements to reconstructions:

$$\hat{x} = f_\theta(y).$$

This uses the same ingredients introduced earlier: layers with learnable parameters, non-linear activations, and training by reducing a discrepancy between the network output $\hat{x}$ and the desired output x on a set of examples.

Because both y and x are images, CNNs are particularly well-suited for this task: convolutions process data on a grid and exploit local spatial structure. A common choice is an **autoencoder** architecture, which transforms the input measurement into intermediate feature maps and then produces an output image of the same size. Such architectures are widely used in tasks like:

- **Denoising**: input is noisy y, target is clean x.
- **Super-resolution**: input is low-resolution y, target is high-resolution x.
- **Deblurring**: input is blurred y, target is sharp x.

In this setting, the network implicitly learns an *image prior* from data, that is, it learns what plausible clean images look like, and uses that knowledge to guide reconstruction.

4 Implementation exercises in PyTorch

Please access the provided `.ipynb` notebook, create a copy, and start programming!

Part A: Linear Layers

The goal of Part A is to implement the core computation behind a perceptron and understand how choices like activation and initialization affect outputs.

Deliverables

- **A1:** Report tensor shapes and verify that your implementation matches `X @ w + b`.
- **A2:** Show a small histogram or min/max summary for outputs with two different activations and explain the difference in one sentence.
- **A3:** Plot the loss vs. iteration and report the learned (w, b) compared to $(w_{\text{true}}, b_{\text{true}})$.

Part B: Multilayer Perceptrons (MLPs)

The goal of Part B is to move from a single linear layer to a multilayer model and observe how depth, activation, and data size affect performance and overfitting.

Deliverables

- **B2:** Report test accuracy and include a confusion matrix for at least 5 classes.
- **B3:** Plot training vs. validation loss curves for (i) no regularization and (ii) one regularization method; include a short explanation.

Part C: Convolutional Neural Networks

The goal of Part C is to replace vectorization with convolutions and observe how respecting spatial structure changes performance and behavior.

Deliverables

- **C1:** Report CNN vs. MLP accuracy in a small table and write 2–3 sentences explaining the difference.
- **C2:** Provide a figure showing multiple feature maps from the first convolutional layer and describe what you observe.
- **C3:** Show examples of: ground truth x, blurred image y, CNN reconstruction $\hat{x}$. Compute PSNR and SSIM for the Blurred image vs. ground truth, and for the CNN reconstruction vs. ground truth.

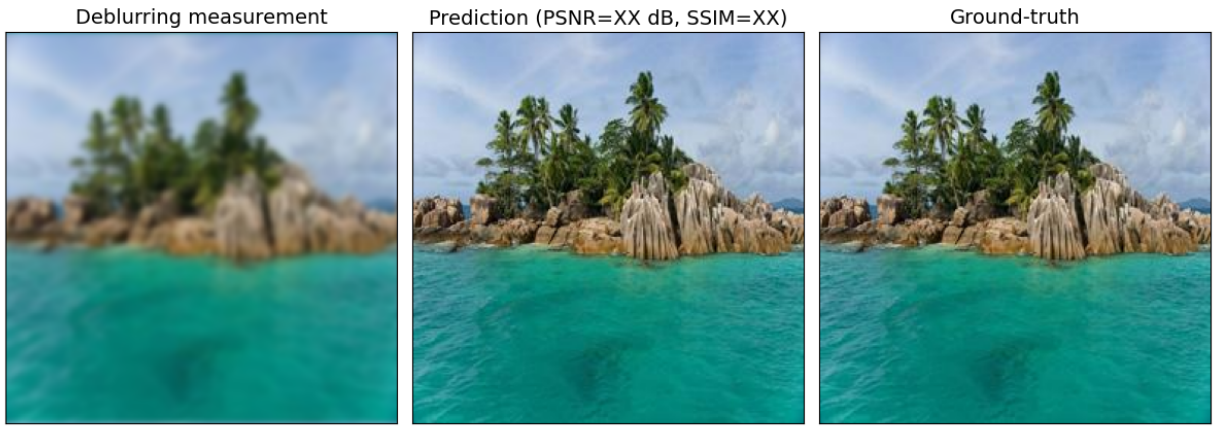


Figure 8: Example of how to report the visual restoration results for Part C (input measurement, network output with PSNR, ground truth).

5 Analysis Questions (short answers)

Answer briefly:

1. Besides standard gradient descent, what other optimization algorithms are commonly used for training neural networks? Briefly describe how they differ conceptually.
2. Why can a model achieve low training loss but poor validation performance?
3. What changes when you replace an MLP with a CNN?
4. What are the feature maps?

6 Final Deliverables

Deliverable	Minimum content
Notebook (.ipynb)	Executable code, intermediate figures, short conclusions per experiment, and a decisions/fixes block.
Report (PDF)	One figure per experiment, requested metrics tables, and clear answers to all the questions posed throughout this document.
Results	`results/` folder, inside a `.zip` file, with exported images.